

Chapter 2: Software metrics, Software project planning and Risk

2.1 Basic terminologies of Software metrics (Measures, Metrics, Indicators):

- **Measure** is a direct, numerical recording of an attribute of a product or process. It's the raw data that you collect. **Eg: The number of lines of code, the number of bugs found in a specific release, or the time taken to complete a task**
- **A Metrics** are quantifiable measures used to track the performance, quality, health, and progress of a project, process, or product. It is often a calculated value derived from one or more measures. They provide **objective, quantifiable data** to measure performance, track progress against objectives, predict future outcomes, and guide **informed decision-making** for control and continuous improvement.
Eg: Calculating defect density by dividing the number of defects (measure) by the lines of code (measure).
- **Indicator** is a metric or a combination of metrics used to provide insight into the overall status or health of a project, product, or process. It helps in decision-making and tracking progress toward a goal. **Eg: A dashboard that shows a green, yellow, or red status for "project progress" based on a combination of metrics like schedule, budget, and bug counts.**
- A **software metric** is a measure of software characteristics which are measurable or countable OR, Software metrics are **quantifiable measures** used to understand, control, and improve the quality, efficiency, size, and performance of software products and development processes. Software metrics are valuable for many reasons, including measuring software performance, planning work items, measuring productivity, and many other uses.
- Software metrics are standards of measurement that are used to measure various attributes of software processes, products, and projects. They help in assessing the efficiency, quality, and productivity of software development efforts and are critical in decision-making throughout the software development lifecycle (SDLC).
- Software metrics are a quantifiable or countable assessment of the attributes of a software product. There are 4 functions related to software metrics:

1. **Planning:** estimating size, cost, effort, and schedule.
2. **Organizing:** allocating resources and assigning tasks.
3. **Controlling:** monitoring progress, quality, and taking corrective actions.
4. **Improving:** enhancing software quality, productivity, and processes.

Characteristics of software Metrics

1. **Quantitative:** Metrics must possess a quantitative nature. It means metrics can be expressed in numerical values.

2. Understandable: Metric computation should be easily understood, and the method of computing metrics should be clearly defined.

3. Applicability: Metrics should be applicable in the initial phases of the development of the software. Metrics must provide useful information from the beginning, including the requirement analysis, planning, and design phases, and not only after coding or testing.

4. Repeatable: When measured repeatedly, the metric values should be the same and consistent.

5. Economical: The computation of metrics should be economical. The computation and collection of software metrics should be cost-effective. The effort, time, and cost required to gather and compute the metric must be lower than the value it provides to the project.

6. Language Independent: Metrics should not depend on any programming language.

Advantages of Software Metrics:

1. Reduction in cost or budget.
2. It helps to identify the particular area for improvising.
3. It helps to increase the product quality.
4. Managing the workloads and teams.
5. Reduction in overall time to produce the product.
6. It helps to determine the complexity of the code and to test the code with resources.
7. It helps in providing effective planning, controlling and managing of the entire product.

Disadvantages of Software Metrics

1. It is expensive and difficult to implement the metrics in some cases.
2. Performance of the entire team or an individual from the team can't be determined. Only the performance of the product is determined.
3. Sometimes the quality of the product is not met with the expectation.
4. It leads to measure the unwanted data which is wastage of time.
5. Measuring the incorrect data leads to make wrong decision making.

2.2 Software metrics guidelines:

I. Define Clear Objectives

- Understand and clearly define why metrics are being collected. Is the goal to improve quality, optimize processes, measure productivity, or assess risk? Well-defined objectives ensure that only relevant metrics are tracked.

II. Select Relevant Metrics

Choose metrics that align with project goals, team objectives, and organizational needs. Choose metrics that match the project's goals.

Example:

For quality improvement → use **defect density, code coverage**.

For productivity → use **effort, velocity**, etc.

III. Focus on Simplicity and Understandability

- Avoid overly complex metrics that are difficult to interpret or communicate. Metrics should be straightforward and easily understood by stakeholders, including developers, managers, and clients.

IV. Ensure Consistency

- Standardize the measurement process to maintain consistency across projects and time periods. For example, define clear rules for counting lines of code or calculating function points so results are comparable and reliable.

V. Ensure Data Accuracy and Validity

- Collect accurate, up-to-date data to maintain trust in the metrics. Implement validation procedures to check data integrity, and update metrics definitions as project requirements or technologies evolve.

VI. Provide Training and Guidance

- Ensure that team members understand the importance of metrics and how to interpret them. Educate stakeholders on the purpose of each metric and how to use them effectively.

2.3 Types of Software metrics:

1. **Product Metrics: Product metrics tell us how good the product is.** Product metrics are used to evaluate the state of the product, tracing risks and uncover prospective problem areas. The ability of the team to control quality is evaluated. Product metrics focus on the characteristics of the software product itself, such as size, quality, and

complexity. These metrics are used to assess the maintainability, reliability, and performance of the software. Common examples include Lines of Code (LOC) for measuring the size, Cyclomatic Complexity for analyzing code structure, and Defect Density to determine the number of defects per unit of software. These metrics help ensure that the final product meets quality standards and is easy to maintain.

2. **Process Metrics: Process metrics tell us how good the development process is.** Process metrics pay particular attention to enhancing the long-term process of the team or organization. Process metrics measure the efficiency and effectiveness of the software development process. These metrics aim to optimize processes, identify bottlenecks, and ensure consistency in producing high-quality software. Examples include effort variance, schedule variance, defect injection rate, and lead time. Process metrics are essential for continuous improvement and maintaining predictable outcomes in software projects.
3. **Project Metrics: Project metrics tell us how well the project is being managed.** Project Metrics are quantitative measures used by a project manager to assess the status of an ongoing project, track potential risks, uncover problem areas, and adjust the work flow to ensure the project is delivered on time, within budget, and to the required quality standard. The project metrics describes the characteristic and execution of a project. Examples include effort estimation accuracy, schedule deviation, cost variance, and productivity. Usually measures-
 - Number of software developer
 - Staffing patterns over the life cycle of software
 - Cost and schedule
 - Productivity

2.4 Objectives, Scope and Resources, project Estimation, Decomposition

Techniques:

- **Objectives, Scope, Resources, Estimation, and Decomposition** are indeed key components of **Software Project Planning**.
- **Software Project Planning** is the process of defining, organizing, estimating, scheduling, and managing all activities required to successfully complete a software project.
- It ensures that the project is completed **on time, within budget, and with desired quality**.

❖ Objectives of software project:

- The objectives define what the project aims to achieve.
- Clearly defined goals for a software project.

- Ensure the final product meets the user's requirements, quality standards, and business expectations.
- Objectives define the **SMART** (Specific, Measurable, Achievable, Relevant, Time-bound) targets for the software product.

Examples of objectives:

- Deliver software on time and within budget.
- Achieve specified performance and functionality.

❖ **Scope of a Software Project:**

- Defines the boundaries and deliverables of the project.
- Includes all functionalities and features to be delivered.
- Helps manage stakeholder expectations and prevent scope creep. (uncontrolled changes to the project)
- Scope helps to estimating resources and timelines accurately.

❖ **Resources**

- Project Resources are all the assets, inputs, and capabilities required to execute a project and achieve its objectives. They are essential for turning a project plan into a deliverable product or service.

Types:

- **Human Resources:** Skilled team members like developers, designers, and testers.
- **Tools and Infrastructure:** Development tools, testing environments, hardware, and software.
- **Financial Resources:** Budget allocated for the entire project.
- **Time:** Duration required for task completion and project delivery

❖ **Project Estimation:**

- ❖ Process of predicting effort, time, and cost required to complete the project. Effective estimation models are essential for setting realistic goals, planning resources, and managing project risks.

Benefits of Project Estimation:

1. Resource Allocation:

- Accurately estimating project size helps determine the necessary budget for personnel, hardware, software, and other resources.

2. Project Scheduling:

- By understanding the scope and complexity of the project, realistic timelines can be created for various stages of development, testing, and deployment.

3. Risk Management:

- Project size estimation helps identify potential risks and challenges that may arise during development.

4. Quality Assurance:

- The estimated project size helps determine the necessary testing effort to ensure the

software meets quality standards.

5. Client Expectations:

- Accurate project size estimation helps set realistic expectations with clients regarding delivery timelines and project scope.

6. Project Management:

- Project size estimation provides a baseline for tracking progress and identifying potential deviations from the plan.

7. Evaluation and Improvement:

- After project completion, comparing the actual effort and resources used to the estimated size helps identify areas for improvement in future projects.

Types of Project Estimation:

Model	Best For	Description
Expert Judgment	Small projects or when expert insights are available	Estimates based on expert opinions and experience
Analogous Estimation	Early-stage projects with similar historical data	Uses data from similar past projects
Parametric Estimation	Quantifiable projects with historical data	Relies on statistical inputs for cost and time estimates
COCOMO	Medium to large projects with well-defined inputs	Structured cost model with basic, intermediate, detailed levels
Function Point Analysis	Business apps with user-centered requirements	Estimates based on user-oriented functionality
Wideband Delphi	Complex projects with multiple experts involved	Collaborative, consensus-based approach
Three-Point (PERT)	Projects with high uncertainty	Uses optimistic, pessimistic, and most likely estimates
Top-Down Estimation	Early-stage high-level estimation	Starts with overall estimate and breaks it down
Bottom-Up Estimation	Detailed projects with defined tasks	Builds up estimates from individual tasks
ML-Based Models	Data-rich organizations with data science expertise	Predicts estimates using machine learning

❖ Decomposition technique

- Decomposition techniques are used to break down the total project work into smaller, more manageable tasks, making estimation easier and more accurate.

Two commonly used decomposition models are:

1. Lines of Code:

- Estimation is done on behalf of number of line of codes in the software product. As the name suggest, LOC count the total number of lines of source code in a project.

The units of LOC are:

KLOC- Thousand lines of code

NLOC- Non-comment lines of code

KDSI- Thousands of delivered source instructions

LOC includes: Declarations, actual code including logic and computation

LOC excludes: Blank lines and comments

Advantages:

- Universally accepted and is used in many models like COCOMO.
- Estimation is closer to developer's perspective.
- Simple to use.

Disadvantages:

- Different programming languages contain different number of lines.
- No proper industry standard exists for this technique.

2. Function Point Analysis (FPA):

- Function Points estimate software size based on the **number and type of functions** the system supports.
- The description of FP is correct in its core concept and its step-by-step process, which is based on the **International Function Point Users Group (IFPUG) method**.
- FP is an **indirect measure** of software size based on user-visible functionality.

The steps you listed are fundamentally correct and represent the logical flow of FP analysis:

1. **Count the number of functions of each proposed type.**

This involves counting and classifying the five primary function types:

- External Inputs (EI),
- External Outputs (EO),
- External Inquiries (EQ),
- Internal Logical Files (ILF),
- External Interface Files (EIF).

2. **Compute the Unadjusted Function Points (UFP):**

UFP is calculated by summing the counts from step 1 after applying complexity weights (Low, Average, High) to each type.

3. **Find Total Degree of Influence (TDI):**

TDI is the sum of the scores of 14 **General System Characteristics (GSCs)**, ranging from 0 to 5 for each.

4. **Compute Value Adjustment Factor (VAF):**

VAF is calculated using the formula: $VAF = 0.65 + (0.01 * TDI)$

5. **Find the Function Point Count (FPC):**

The final Function Point Count is calculated as: $FPC = UFP * VAF$

Advantages:

- Language-independent.
- Can be used early in SDLC.
- Focuses on user functionality.
- More accurate than LOC for business systems.

Disadvantages:

- Calculation is complex and time-consuming.
- Subjective judgments affect accuracy.
- Not suitable for real-time/scientific systems.
- Needs clear requirements.

2.5 Empirical Estimation Models, Risk Management Strategies:

❖ Empirical Estimation Models:

- Empirical estimation techniques are based on the data taken from the previous project and some based on guesses and assumptions.
- Empirical estimation is a technique or model in which empirically derived formulas are used for predicting the data that are a required and essential part of the software project planning step.
- It uses the size of the software to estimate the effort.
- In this technique, an educated guess of project parameters is made.
- Hence, these models are based on common sense

Two most common type of Empirical Project Estimation Techniques are:

1. **Expert Judgment Technique**
2. **Delphi Cost Estimation Technique**

1. Expert Judgement Technique:

- Expert judgment is a technique that relies on the knowledge and experience of domain experts to estimate project costs, schedules, and risks.
- It's a valuable tool when historical data is limited or unavailable.

How it works:

1. Identify Experts: Select experts with relevant experience and knowledge in the specific domain.

2. Gather Estimates: Collect individual estimates from the experts, either through interviews, surveys, or workshops.

3. Consolidate Estimates: Analyze the estimates and identify any significant variations or outliers.

4. Refine Estimates: Discuss the estimates with the experts to refine them and reach a Consensus (Group agreement).

5. Adjust for Risk: Consider potential risks and uncertainties and adjust the estimates accordingly.

Advantages of Expert Judgment:

- **Leverages Expertise:** It taps into the knowledge and experience of domain experts.
- **Quick and Efficient:** It can be a relatively quick and efficient technique.
- **Flexibility:** It can be applied to a wide range of estimation problems.

Disadvantages of Expert Judgment:

- **Subjectivity:** Estimates can be influenced by individual biases and opinions.
- **Lack of Objectivity:** It relies on human judgment, which can be prone to errors.
- **Limited Accuracy:** The accuracy of estimates may vary depending on the expertise and experience of the experts.

When to Use Expert Judgment:

- When there is limited historical data or statistical information.
- When the project is novel or innovative.
- When quick and rough estimates are needed.

2. Delphi Cost Estimation Technique

- The Delphi Method is a structured technique used to elicit expert opinions on a particular topic.
- In the context of software engineering, it's used to estimate project costs, schedules, and risks.

How it works:

- 1. Selection of Experts:** A group of experts with relevant knowledge and experience is selected.
- 2. Initial Questionnaire:** A questionnaire is sent to the experts, asking them to provide their individual estimates.
- 3. Feedback and Iteration:** The responses are collected, analysed, and shared with the group. Experts are encouraged to review their estimates based on the feedback from others.
- 4. Consensus Building:** The process of iteration and feedback continues until a consensus is reached or a stable estimate is obtained.

Advantages of Delphi Method:

- **Reduced Bias:** By keeping responses anonymous, it reduces the influence of dominant personalities and groupthink.
- **Structured Approach:** The structured process ensures that all relevant factors are considered.
- **Consensus Building:** It facilitates the convergence of opinions and helps to reach a shared understanding.
- **Flexibility:** It can be adapted to various estimation problems, such as cost, schedule, and risk.

Disadvantages of Delphi Method:

- **Time-Consuming:** The iterative process can be time-consuming, especially if there are many experts involved.
- **Expert Availability:** It requires the availability and cooperation of experts.
- **Subjectivity:** While it reduces bias, it still relies on expert judgment, which can be subjective.
- **Difficulty in Reaching Consensus:** In some cases, it may be difficult to reach a consensus,

especially if there are strong disagreements among experts.

When to Use Delphi Method:

- When there is a lack of historical data or reliable statistical models.
- When expert judgment is crucial for making accurate estimates.
- When you want to reduce the influence of individual biases.

❖ **Risk Management Strategies:**

Risk is the effect of uncertainty on objectives. It's an uncertain event or condition that, if it occurs, has a positive or negative effect on one or more goals (time, cost, scope, quality, reputation, etc.).

OR, A Risk is an event that has not happened yet, but could happen, and if it does, it will affect your goals.

Risk Management Strategies are the planned approaches used to deal with project risks in order to reduce their negative impact or take advantage of potential opportunities.

★ **Risk management process:**

- Risk management is a sequence of steps that help a software team to understand, analyze, and manage uncertainty.

Risk management process consists of

- Risks Identification.
- Risk Assessment.
- Risks Planning.
- Risk Monitoring.



Risk Management Process

1. Risk Identification:

- Risk identification refers to the systematic process of recognizing and evaluating potential threats or hazards that could negatively impact an organization, its operations, or its workforce. This involves identifying various types of risks, ranging from IT security threats like viruses and phishing attacks to unforeseen events such as equipment failures and extreme weather conditions.

2. Risk analysis:

- Risk analysis is the process of evaluating and understanding the potential impact and likelihood of identified risks on an organization. It helps determine how serious a risk is and how to best manage or mitigate it. Risk Analysis involves evaluating each risk's probability and potential consequences to prioritize and manage them effectively.

3. Risk Planning:

- Risk planning involves developing strategies and actions to manage and mitigate identified risks effectively. It outlines how to respond to potential risks, including prevention, mitigation, and contingency measures, to protect the organization's objectives and assets.

4. Risk Monitoring:

- Risk monitoring involves continuously tracking and overseeing identified risks to assess their status, changes, and effectiveness of mitigation strategies. It ensures that risks are regularly reviewed and managed to maintain alignment with organizational objectives and adapt to new developments or challenges.

Benefits of risk management:

Here are some benefits of risk management:

- Helps protect against potential losses.
- Improves decision-making by considering risks.
- Reduces unexpected expenses.
- Ensures adherence to laws and regulations.
- Builds resilience against unexpected challenges.
- Safeguards company reputation.

Limitation of Risk Management:

Here are Some Limitation of Risk Management

- Too much focus on risk can lead to missed opportunities.
- Implementing risk management can be expensive.
- Risk models can be overly complex and hard to understand.
- Relies on accurate human judgment and can be prone to mistakes.
- Some risks are hard to predict or quantify.
- Managing risks can take a lot of time and resources.

★ Risk management strategies:

➤ They are the methods used to handle risks, such as avoiding, reducing, transferring, or accepting them. There are four types of strategies of risk management. They are:

1. Risk Avoidance:

- The goal is to eliminate the risk completely.
- This is done by changing the project plan, requirements, or approach so the risk cannot occur.
- Taking proactive steps to remove the cause of the risk entirely, such as changing the project scope or substituting a new technology with a proven one.

2. Risk Mitigation (Reduction):

- Reduce the likelihood (chance that risk will occur) or impact of the risk.
- Preventive actions are taken before the risk occurs.

- Implementing control measures to lower the risk to an acceptable level, such as conducting rigorous testing, prototyping, or providing additional training.

3. Risk Transfer:

- The risk responsibility is shifted to a third party.
- The risk remains, but someone else handles its impact.
- Moving the risk to a third party through contractual agreements, purchasing insurance, or outsourcing a risky component.

4. Risk Acceptance:

- The project team decides to accept the risk without taking action.
- Making a conscious decision to tolerate the risk, either with or without a back-up plan.
- The project team knows the risk exists and understands what could happen, but decides **not to take any action** because it's not needed or not practical.

★ Risk Management Approaches (types):

- Risk Management Approaches are broad methods or mindsets used to handle risks in a project or organization. They describe how and when risks are addressed, rather than the specific actions taken.

Main Types:

I. Proactive Risk Management:

- In this approach, risks are identified, analyzed, and mitigated before they occur. The focus is on planning and prevention.

Key Characteristics:

- Anticipates risks in advance.
- Focuses on minimizing or eliminating risks.
- Involves comprehensive planning and risk mitigation strategies.

Example:

- During software development, the team anticipates that integrating a new project may fail. They create a backup plan, such as using an alternative Project, before starting development.

II. Reactive Risk Management:

This approach involves addressing risks or problems after they have occurred. The focus is on damage control and recovery.

Key Characteristics:

- Responds to risks as they arise.
- Emphasizes quick problem-solving and minimizing the impact of issues.
- Often less planned and more immediate.

Example:

- During module integration, a failure occurs unexpectedly.
- The team fixes the problem immediately and tries to limit delays.

Comparison of Proactive vs Reactive Risk Management

Aspect	Proactive Risk	Reactive Risk
Timing	Before the risk occurs	After the risk occurs
Focus	Prevention and planning	Resolution and recovery
Cost	Usually lower, as prevention is less expensive	Often higher, due to unplanned efforts
Mindset	Anticipatory and strategic	Responsive and tactical
Examples	Creating contingency plans, stress-testing	Debugging production failures, applying patches
Advantages	Fewer disruptions, improved confidence	Flexibility to handle unexpected situations
Disadvantages	May require time and resources upfront	Risk of higher losses or delays

2.6 Software Risks, Risk Identification, Risk Projection

❖ Software Risks:

- Software risks are potential events or conditions that could negatively impact the successful development, deployment, or maintenance of a software project.
- These risks can manifest in various forms, leading to delays, cost overruns, quality issues, or even project failure.

Types of Software Risks:

There are diff types of risks that may affect a computer code project:

1. Project Risks:

These risks threaten the project plan. Project risks identify potential budgetary, schedule, personnel (staffing and organization), resource, stakeholder, and requirements problems and their impact on a software project.

2. Technical Risks:

Technical risks concern potential style, implementation, interfacing, testing, and maintenance issues. Technical risks are problems that can occur in a project because of technology. They include unclear or incomplete requirements, requirements that keep changing, uncertainty about technical solutions, and technology becoming outdated. These risks often happen because the project team does not have enough information about the project.

3. Business Risks:

This type of risk embodies the risks of building a superb product that nobody needs, losing monetary funds or personal commitments, etc. This risk threaten the feasibility. It includes the top five business risks which are:

1. Market risk: system that no one really wants
 2. Strategic risk: product no longer fits into the overall business strategy
 3. Sales risk: the sales force doesn't understand how to sell the product
 4. Management risk: losing the support of senior management
 5. Budget risks: losing budgetary or personnel commitment
-
4. **Known risks:** Risk that can be uncovered after careful evaluation of the project plan, the business and technical environment in which the project is being developed, and other reliable information sources (e.g., unrealistic delivery date, lack of documented requirements or software scope, poor development environment).
 5. **Predictable risks:** These risks are identified from past project experience (e.g., staff turnover, poor communication with the customer, dilution of staff effort as ongoing maintenance requests are serviced).
 6. **Unpredictable risks:** They can and do occur, but they are extremely difficult to identify in advance.

❖ **Risk Identification:**

- Risk identification is the first steps in software engineering risk management.
- it is the process of systematically identifying and documenting potential risks that could negatively impact a project's objectives.
- It focuses on finding what could go wrong, including technical, financial, organizational, and external risks, before they occur, so that they can be analyzed and managed effectively.

Techniques for Risk Identification:

a. Brainstorming:

- Gather the project team, stakeholders, and relevant subject matter experts (SMEs) in a session. Encourage a free-flow of ideas where every potential risk is written down without immediate judgment.

b. Checklists:

- Using pre-developed lists of risks, often based on previous, similar projects or industry-standard risk categories (e.g., technical, external, organizational).

c. Expert Interviews / Delphi Technique:

- Conduct structured, semi-structured, or open-ended interviews with individuals/ groups who have specific experience in similar projects, technologies, or business areas.

d. SWOT Analysis:

- A structured exercise where the team identifies the project's internal **Strengths** and **Weaknesses**, and external **Opportunities** and **Threats**.

e. Document Reviews:

- Document Review is a technique where project, business, and historical records are systematically examined to identify risks that are embedded in or hinted at by the available written information.

❖ **Risk Projection:**

- Risk projection in software engineering is the process of estimating the probability and impact of potential risks identified during the risk analysis phase.
- The ultimate goal is to proactively manage risks before they negatively affect a project.
- It is also called the Risk estimation.

Four Main Activities:

1. Establishing a probability scale

This activity involves assigning a likelihood value to each identified risk, typically expressed as a numerical probability (ranging from 0 to 1) or as qualitative categories such as low, medium, or high.

Example: The probability of failure of a third-party software module is estimated at 0.3 (30%).

2. Defining risk consequences:

In this step, the potential impact of each risk is analyzed to determine what would occur if the risk happen. The consequences may affect various project parameters, including cost, schedule, quality, or system performance.

Example: Failure of the module could result in a delay of two weeks in the project timeline or incur \$5,000 in rework costs.

3. Estimating the project impact:

Evaluate how the risk could affect cost, schedule, quality, or scope.

Estimating the project impact involves evaluating the potential consequences of a risk and translating them into a quantifiable value, often denoted as the impact *C*.

This step enables the final calculation of **Risk Exposure (RE)**:

$$\text{RE} = \text{Probability (p)} * \text{Impact (c)}$$

Where , **Probability (P)**: Likelihood that the risk will occur (0 to 1 or 0–100%)

Impact (C): The potential effect on the project (cost, time, quality, etc).

4. Maintaining accuracy:

To ensure reliability, risk projections should be based on accurate and credible (trustworthy) data, including historical project information and expert judgment. Projections must be continuously updated as project conditions change, ensuring that risk management decisions remain realistic and effective.

Importance of Risk Projection:

I. Prioritizes Risks

- By estimating probability and impact, risk projection helps identify which risks are most serious and require immediate attention.
- Resources and efforts can be focused on high-priority risks.

II. Supports Decision-Making

- Provides quantitative or qualitative data to guide risk response planning.
- Helps project managers decide whether to mitigate, transfer, accept, or avoid each risk.

III. Facilitates Resource Allocation

- Knowing the potential impact allows the project team to allocate budget, time, and manpower effectively for risk mitigation.

IV. Enhances Planning Accuracy

- By understanding potential delays, costs, or quality issues, risk projection improves project schedules and budgets.

V. Reduces Project Uncertainty

- Provides foresight into possible challenges, allowing teams to prepare in advance and reduce surprises.

VI. Enables Quantitative Analysis

- Supports calculation of Risk Exposure ($RE = Probability \times Impact$), which is critical for quantitative risk assessment.

Assignments

1. Define Software Metrics and briefly explain the difference between Measures, Metrics, and Indicators.
2. Explain how each types of software metrics helps in improving software quality and productivity. Differentiate between product metrics and process metrics.
3. What is Software Risk? & its types. Differentiate between Risk Identification and Risk Projection
4. Explain software project planning.
5. What is cost estimation technique in software engineering? Explain its types.
6. Illustrate the difference between expert judgment and Delphi cost estimation technique.
7. Explain the Risk Management Process in software projects with examples.
8. What is LOC? Explain the steps of function point Analysis.
9. Explain **COCOMO model**. Also mention its strengths and weaknesses.
10. Solve the all numerical related to function point and LOC from past question. (2020-2025)

